Real-Time Uncertainty-Aware Grasping Pipeline for Edge-Accelerated Robotic Arms

Primary Track: F (Applied AI Systems: Perception and Robotics) Secondary Track: E (calibrated uncertainty with a rule-based safety layer)

Team No.: 34

Team Member

Reg. No.	Name	Primary role in the project
24BCT0062	Siddhant Kumar	Entire project (single-member team)

Report submitted for the

Final Project Review of

Course Code: BCSE306, Artificial Intelligence

Slot: C1

Professor: Dr. Ilanthenral Kandasamy

GitHub Repository: https://github.com/RPi-Boy/Automator

Demo Video: Google Drive

Vellore Institute of Technology, Vellore

Fall Semester 2026-27 | October 2026

Declaration and Individual Contribution

I declare that this report and the accompanying code are my own work, carried out for BCSE306 Artificial Intelligence during Fall Semester 2026-27. All sources, datasets, pre-trained models and AI tools used have been acknowledged. Every reference cited has been checked by me against its original source. I understand that I may be examined on any part of this work during the viva.

Reg. No.	Name	Modules / components owned	Report sections written	Contribution (%)	Signature
24BCT0062	Siddhant Kumar	Simulation scene and dataset generator; object finder; perception network and uncertainty calibration; inverse kinematics and trajectory planner; ROS 2 nodes; ESP32 firmware; SAC reach benchmark; all evaluation scripts	All sections	100	

Abstract

Robot arms guided by a camera and a neural network can make confident mistakes on objects they were never trained on. In this project I built a shape-sorting system for a four-joint robot arm that checks how familiar each object looks and stops the arm when an object looks unfamiliar. Background subtraction finds every object on the table, whatever its shape. A small MobileNetV3 network then names each object (sphere, cube, cylinder or brick), corrects its position, and gives it an unfamiliarity score based on the Mahalanobis distance to the four known shapes. The stop threshold is set so that only about 1% of known objects trigger it. The arm, simulated in PyBullet, then carries each object to its tray using exact inverse kinematics and smooth joint paths. On 2,217 test images the network named the shape correctly 99.91% of the time and located objects within 1.97 mm on average, against 5.43 mm from camera geometry alone. The arm sorted 74 of 80 objects (92.5%) into the right tray; the first version of the project sorted none. With an unseen object on the table, the arm stopped in 25 of 30 trials, every time the object was detected. On a Jetson Orin Nano, finding, checking and planning take 5.91 ms per frame with TensorRT, at about 7 W. The main weakness is unseen objects that look like a known shape: only 13.6% of capsules were caught. The motor controller and real arm are not tested yet.

Keywords: robotic sorting; unknown-object detection; Mahalanobis distance; MobileNetV3; TensorRT; edge AI

Table of Contents

Declaration and Individual Contribution....2
Abstract....3
Table of Contents....4
1. Introduction....6
1.1 Background and Motivation....6
1.2 Problem Statement....6
1.3 Research Questions / Hypotheses....6
1.4 Contributions....7
1.5 Organisation of the Report....7
2. Literature Review....7
2.1 Search Strategy....7
2.2 Literature Summary Table....8
2.3 Synthesis and Research Gap....10
3. Objectives of the Project....10
4. Innovation Component and System Design....11
4.1 Innovation Component....11
4.2 Positioning Against the Closest Existing Work....12
4.3 System Architecture....12
4.4 Module Description....13
4.5 AI Techniques Used and Design Rationale....14
5. Methodology and Implementation....17
5.1 Data....17
5.2 Preprocessing....17
5.3 Models Used....18
5.4 Prompting, Fine-tuning and Retrieval Configuration....18
5.5 Experimental Setup and Hyperparameters....18
5.6 Compute and Cost....19
5.7 Tools and Frameworks....20
5.8 Implementation, Screenshots and Demo....20
6. Evaluation Protocol....21
6.1 Evaluation Data and Split Hygiene....21
6.2 Metrics....21
6.3 Baselines....22
6.4 Human Evaluation and LLM-as-a-Judge (if used)....22
6.5 Statistical Analysis....22
7. Results and Discussion....23

7.1 Main Results....23
7.2 Comparative Analysis....26
7.3 Ablation Study....26
7.4 Error Analysis....27
7.5 Answers to the Research Questions....28
7.6 Threats to Validity and Limitations....29
8. Responsible AI and Ethical Considerations....29
8.1 Risk Assessment....29
8.2 Data Privacy and Consent....30
8.3 Fairness....30
8.4 Safety, Security and Misuse....30
8.5 Licences and Attribution....31
9. Conclusion and Future Work....31
10. Reproducibility Checklist....32
11. References....32
12. Declaration of Generative AI Use....33
Appendices....35

List of Figures

Fig. 4.1: System architecture and data flow....13
Fig. 7.1: Confusion matrix on 2,217 held-out crops....23
Fig. 7.2: Time per frame for each setup, mean and 95th percentile....25
Fig. 7.3: Sorting success as each change was added....27

List of Tables

Table 2.1: Summary of reviewed literature....8
Table 3.1: Project objectives and how they are measured....10
Table 4.1: Comparison with the three closest existing works....12
Table 4.2: AI techniques used in the project....14
Table 5.1: Dataset description....17
Table 5.2: Model inventory....18
Table 5.3: Hyperparameters and settings....18
Table 5.4: Tools used and justification....20
Table 7.1: Main results (single training run; 95% CI in brackets; best value in bold)....23
Table 7.2: Unknown-object detection by split and object type....24
Table 7.3: Safety test with one unseen object among three known objects....24
Table 7.4: Time per frame for each setup (ms, 3 objects per frame)....25
Table 7.5: Sustained 60 s test on the Jetson Orin Nano Super....25
Table 7.6: Ablation, effect of removing each component....26
Table 7.7: Representative failure cases....28
Table 8.1: Risks and mitigations....30
Table 12.1: Generative AI tools used....34

1. Introduction

1.1 Background and Motivation

Robot arms are no longer limited to fenced-off factory cells. They now work in warehouses, recycling plants and labs, often close to people. Most of them decide what to do using a camera and a trained neural network. The problem is that a network always gives an answer, even for an object it has never seen, so an arm could pick up a tool, a broken part or someone's hand as if it were one of its known objects. Recent sorting and grasping systems report high success rates [6]-[8], but they treat everything they see as one of their known classes.

Spotting inputs that are unlike the training data is called out-of-distribution (OOD) detection, and it is an active research area [1]-[3]. Some recent robot work already uses uncertainty to decide when to ask a person for help [5]. Small boards such as the NVIDIA Jetson Orin Nano are now fast enough to run these checks right next to the arm. In this project I combine these ideas into a low-cost sorting arm that knows when to stop. It is meant for small sorting lines, teaching labs and hobby builders who want a cheap arm that is safe to work near.

1.2 Problem Statement

A camera looks at a table with known objects (spheres, cubes, cylinders and bricks) and possibly some unknown ones. A four-joint arm has to put each known object in its own tray and must not touch anything it does not recognise. Each decision has to fit well within the 30 ms per frame set in my first review, so that it can run on a small edge computer. The arm should also rarely stop by mistake on a known object; otherwise it would not be useful.

1.3 Research Questions / Hypotheses

Since this is a system-building project, I tested five hypotheses:

- **H1.** Background subtraction followed by a MobileNetV3 network can name the four shapes with at least 99% accuracy and locate them within 5 mm on average, on simulated test images.
- **H2.** A Mahalanobis-distance score, tuned so that 1% of known objects go over the threshold, can tell unseen objects from known ones with an AUROC of at least 0.95, while wrongly stopping on at most 10% of known objects when 95% of unknowns are caught.
- **H3.** Together with exact inverse kinematics, the system can sort known objects in simulation and stop whenever an unseen object is on the table.

- **H4.** A Soft Actor-Critic (SAC) reinforcement learning agent can learn to bring the fingertip within 1 cm of a target, as an alternative to inverse kinematics.
- **H5.** On a Jetson Orin Nano, finding objects, running the network and planning the move fit within 30 ms per frame, and TensorRT makes this faster than the board's CPU.

1.4 Contributions

- A complete simulation of the sorting task, and a dataset generator that keeps unseen objects aside for testing (Section 5.1).
- One exported model that gives the shape, a position correction and a calibrated unfamiliarity score (Sections 4.5 and 7.1).
- A stop chain that runs from the network down to the motor controller, tested in 30 trials with unseen objects (Table 7.3).
- A check of the first version of the project, which sorted 0% of objects, and a rebuilt version that sorts 92.5% (Sections 7.2 and 7.3).
- Measurements on a Jetson Orin Nano Super: 5.91 ms per frame with TensorRT against 11.25 ms on its CPU, steady over 60 s at about 7 W (Table 7.4).
- A direct comparison of learned reaching (SAC) with inverse kinematics (Section 7.1).

1.5 Organisation of the Report

Section 2 reviews related work, Section 3 sets objectives, Section 4 explains the design, Section 5 the data and tools, and Section 6 the testing. Section 7 gives the results. Sections 8 to 12 cover responsible AI, conclusions, reproducibility, references and my use of AI tools.

2. Literature Review

2.1 Search Strategy

I searched Google Scholar, Semantic Scholar, IEEE Xplore and arXiv for papers from January 2023 to October 2026. Keywords included "out-of-distribution detection", "Mahalanobis distance", "uncertainty-aware grasping", "robotic sorting", "open-set detection" and "edge inference". I kept papers that describe a method and report measured results on unknown-object detection, robot picking or sorting, fast vision models, or arm control. I left out papers with no evaluation and papers older than 2023, except the original papers for methods I use directly [15]-[20]. I included 14 papers and checked each one against its official page. Table 2.1 summarises them.

2.2 Literature Summary Table

Table 2.1: Summary of reviewed literature

#	Authors, Year [Ref.]	Title and Venue (journal / conference / preprint)	Problem, Concept or Framework	Methodology / Implementation	Dataset and Evaluation Metrics	Key Findings (with numbers)	Limitations / Gaps Identified	Relevance to Our Project
1	Zhang et al., 2024 [1]	OpenOOD v1.5: Enhanced benchmark for out-of-distribution detection. J. Data-centric ML Research, 2024	Fair comparison of unknown-input detectors	One code base testing about 40 methods on near and far unknown sets	CIFAR-10/100, ImageNet-200/1K; AUROC, FPR at 95% TPR	No method wins on all benchmarks; best near-OOD AUROC on ImageNet-1K is 82.16%	Image classification only; no robot or cost of a wrong stop	I adopt its metrics and its near versus far split (capsule versus random meshes)
2	Müller and Hein, 2025 [2]	Mahalanobis++: Improving OOD detection via feature normalization. ICML 2025	Why Mahalanobis scores work well on some networks and badly on others	Normalise feature length before fitting class means and shared covariance	44 ImageNet models; OpenOOD sets; AUROC, FPR	Normalisation improves the Mahalanobis score consistently across models	Large pretrained models; no edge or robot test	A direct fix to try for my weak capsule result
3	Anthony and Kamnitsas, 2023 [3]	On the use of Mahalanobis distance for OOD detection with neural networks for medical imaging. arXiv preprint, 2023	Which network layer to measure the distance at	Compare distances at several depths and combine detectors	CheXpert chest X-rays with unseen pacemakers; AUROC	Best layer depends on the type of unknown input; multi-depth detectors are more robust	Medical images; no timing results	Suggests earlier-layer features for look-alike objects
4	Shi et al., 2024 [4]	vMF-Contact: Uncertainty-aware evidential learning for probabilistic contact-grasp in noisy clutter. arXiv preprint, 2024	Grasp uncertainty in clutter without slow ensembles	Evidential learning with a von Mises-Fisher model of grasp direction	Real cluttered scenes with unseen objects; clearance rate	39% higher clearance rate than baselines	Needs point clouds and a GPU; no stop behaviour	Shows single-pass uncertainty is enough; my score is also single-pass
5	Ren et al., 2023 [5]	Robots that ask for help: Uncertainty alignment for large language model planners. CoRL 2023	When a robot should stop and ask	Conformal prediction over plan options; ask a human when more than one option remains	Simulated and real robots, several ambiguity types; success and help rates	Meets a chosen success level with less human help than ensemble and prompt baselines	Uncertainty in language plans, not in perception	Same idea as my calibrated stop; theirs has a formal guarantee, mine is measured on data
6	Fang et al., 2023 [6]	AnyGrasp: Robust and efficient grasp perception in spatial and temporal domains. IEEE T-RO 39(5), 2023	Grasping any object quickly	Learned 7-DoF grasp poses from depth images, tracked over time	Bins with 300+ unseen objects; success rate, picks per hour	93.3% success; more than 900 picks per hour	Grasps unknown objects instead of refusing them	Contrast: my arm deliberately refuses unknown objects

#	Authors, Year [Ref.]	Title and Venue (journal / conference / preprint)	Problem, Concept or Framework	Methodology / Implementation	Dataset and Evaluation Metrics	Key Findings (with numbers)	Limitations / Gaps Identified	Relevance to Our Project
7	Sun et al., 2024 [7]	Object recognition and grasping for collaborative robots based on vision. Sensors 24(1):195, 2024	Recognise and grasp varied objects	YOLOv3 for class and GG-CNN for grasp pose, run in parallel	COCO, Cornell and own dataset; accuracy, speed	94% accuracy (4.0 points above YOLOv3 alone); 14.1% faster	Closed set: an unknown item is forced into a known class	Same detect-then-pick structure; I add an unknown check
8	Goorun et al., 2024 [8]	Robotic waste sorting using deep learning. MATEC Web Conf. 406:10003, 2024	Sort waste by type with a robot arm	Deep learning classifier, stereo camera and manipulator	Waste items; abstract reports no numbers	Working camera-to-bin sorting system	Closed set; no handling of unexpected items	Closest application; motivates safe handling of unknown items
9	Qin et al., 2024 [9]	MobileNetV4: Universal models for the mobile ecosystem. ECCV 2024	Fast image models across many edge chips	New building blocks searched for mobile accelerators	ImageNet-1K; accuracy, latency	87% top-1 at 3.8 ms on Pixel 8 EdgeTPU	Classification only	Possible upgrade of my MobileNetV3-small backbone
10	Wang et al., 2024 [10]	YOLOv10: Real-time end-to-end object detection. NeurIPS 2024	Real-time detection without post-processing step (NMS)	Dual label assignment and efficiency-driven design	COCO; AP, latency	YOLOv10-S is 1.8x faster than RT-DETR-R18 at similar AP	Finds only trained classes	Learned-detector alternative; would miss unknown shapes my finder catches
11	Liu et al., 2024 [11]	Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. ECCV 2024	Detect objects described in free text	Transformer detector fused with a language model	COCO zero-shot, ODinW; AP	52.5 AP on COCO without COCO training; 26.1 mean AP on ODinW	Heavy for edge devices; needs a text name for each object	Open-set option for finding unknown objects (future work)
12	Zitkovich et al., 2023 [12]	RT-2: Vision-language-action models transfer web knowledge to robotic control. CoRL 2023	Use web-scale knowledge for robot control	Vision-language model fine-tuned to output robot actions	About 6,000 real-robot trials; success rate	Better generalisation to new objects and instructions than earlier robot policies	Very large model; no explicit uncertainty	Large-model end of the design space; my system targets small hardware
13	Tang et al., 2025 [13]	Deep reinforcement learning for robotics: A survey of real-world successes. AAAI 2025	Where deep RL works on real robots	Survey and taxonomy of real deployments	Published deployments	Successes cluster in locomotion and some manipulation; sample efficiency and safety still limit RL	Survey; no new experiments	Explains why my SAC agent is a benchmark and IK drives the arm
14	Pandi et al., 2026 [14]	Experimental validation of jerk-minimized trajectory planning in a 4-DOF industrial robotic arm using inverse kinematics with root multiplicity. Industrial Robot, 2026	Smooth motion for 4-DOF arms	IK with root multiplicity and segmented polynomial profiles on an Arduino arm	Pick-and-place tests; peak jerk, path accuracy	87% lower peak jerk and 64% better path accuracy than trapezoidal point-to-point motion	Low-load tests on low-cost hardware only	Supports my smooth (quintic) joint paths on a 4-DOF arm

2.3 Synthesis and Research Gap

Themes. The papers fall into four groups. (a) Unknown-object detection [1]-[3] is well studied on image datasets. Distance-based methods such as Mahalanobis are cheap and work well, but results change with the network and the kind of unknown object, and very little of this work has been tried on robots. (b) Uncertainty-aware robotics [4], [5] uses uncertainty to pick better grasps or to ask a person for help, but needs depth sensors, GPUs or large language models. (c) Picking and sorting systems [6]-[8] reach 93 to 94% success, but they assume every object belongs to a known class. (d) Fast detectors [9], [10] only find classes they were trained on, while open-set detectors and large robot models [11], [12] are too heavy for a small board. Reinforcement learning [13] and smooth motion planning [14] cover the control side.

Gap. None of these systems combines a shape-independent object finder, a calibrated unfamiliarity score and a hardware stop in something small enough for an edge board. **Gap statement:** there is no low-cost sorting arm that checks whether each object is known, with a known false-stop rate, and refuses to handle the ones that are not.

3. Objectives of the Project

Table 3.1 lists the objectives that follow from this gap. The system has to sort known shapes accurately, catch unknown objects even though nothing was trained on them, rarely stop by mistake, and stay within a time budget that suits an edge board.

Table 3.1: Project objectives and how they are measured

#	Objective (specific and measurable)	Measured by (metric / evidence)	Target or success criterion	Reported in Section
O1	Classify the four shapes from camera crops	Accuracy on 2,217 held-out crops	At least 99%	7.1
O2	Locate each object on the table	Mean 3D position error (mm)	At most 5 mm	7.1
O3	Flag objects never seen in training	AUROC; false-positive rate at 95% true-positive rate	AUROC at least 0.95; FPR at most 10%	7.1
O4	Keep false stops on known objects rare	Share of known test crops above the 0.75 threshold	About 1% (set by calibration)	7.1
O5	Sort known objects in simulation	Objects in the correct tray over 20 trials of 4 objects	Sort by shape (no number fixed in advance)	7.1, 7.3
O6	Stop when an unseen object is present	Trials with a stop on the unseen object, out of 30	Stop in every trial	7.1

#	Objective (specific and measurable)	Measured by (metric / evidence)	Target or success criterion	Reported in Section
O7	Test learned reaching (SAC) against IK	Episodes ending within 1 cm, out of 100	Reach within 1 cm	7.1
O8	Fit an edge time budget	Mean and 95th percentile time per frame	Under 30 ms; then measure on Jetson	7.1

4. Innovation Component and System Design

4.1 Innovation Component

The main idea is simple: **the arm should only touch what it recognises, and I should know how often it will stop by mistake.** Three parts make this work:

- **An object finder that does not care about shape.** Each camera image is compared with a picture of the empty table, so any new object shows up, known or not. The network then only has to judge one cropped object at a time.
- **A calibrated unfamiliarity score built into the model.** After training, I store the average features of each shape and how much they vary. The score measures how far a new object is from the nearest shape, scaled so that 0.75 is the point that only 1% of known validation objects pass. This calculation is part of the exported model file, so the Jetson runs it with no extra code, and its results match the PC version to within 0.005.
- **One way to stop.** A high score, a stop message, no new command for 200 ms, or the hardware stop button all trigger the same halt on the ESP32, which stays on until it is reset.

Getting this to work was harder than it looks. In early versions, tray edges and nearby objects inside the crops pushed false stops up to 7.5% in one part of the image; two touching objects merged into one blob that was always flagged; and the moving arm itself was flagged as unknown. Each needed its own fix, measured in Section 7.3. Table 4.1 compares the design with the three closest systems.

4.2 Positioning Against the Closest Existing Work

Table 4.1: Comparison with the three closest existing works

Feature	vMF-Contact [4]	KnowNo [5]	AnyGrasp [6]	Our project
Uncertainty estimate	Yes (evidential, on grasp direction)	Yes (conformal, on plan choice)	Grasp quality score only	Yes (Mahalanobis, per object)
Action when uncertain	Picks a better grasp	Asks a human	Not applicable	Latched emergency stop
Calibrated threshold	No	Yes, with formal guarantee	No	Yes, measured on data (99th percentile)
Objects never seen in training	Grasped	Not studied	Grasped	Refused; arm stops
Tested on a real robot	Yes	Yes	Yes	No (simulation only)
Compute	GPU	Cloud language model	GPU	Jetson Orin Nano Super, 5.91 ms per frame

4.3 System Architecture

Fig. 4.1 shows how data moves through the system. The simulated camera sends images to the edge computer (a Jetson Orin Nano Super in my tests). For each object, the finder and the network give a position, a shape and an unfamiliarity score u. If u is above 0.75, the planner sends a stop; otherwise it plans the pick. Movement and stop commands go to the simulated arm and, over a USB serial link using micro-ROS, to the ESP32 motor controller.

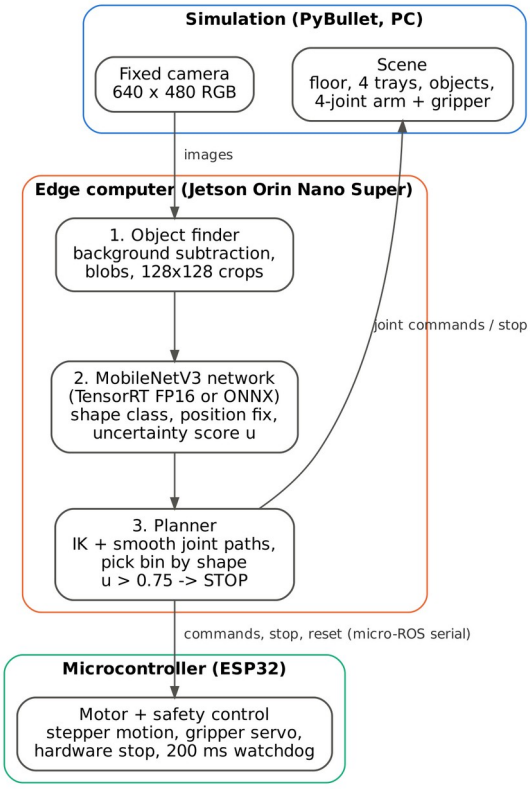


Fig. 4.1: System architecture and data flow

4.4 Module Description

I designed and built all modules myself.

- **Simulation scene** (PyBullet). Creates the floor, four trays with rims, 1 to 4 objects, the four-joint arm with its gripper, and 640 x 480 camera images. Rolling friction stops released objects from rolling away.
- **Object finder** (OpenCV, NumPy). Compares the current image with the empty-table image and gives, for each object, a 128 x 128 crop with the background masked out, a bounding box and a first position estimate on the table.
- **Perception network** (PyTorch, exported to ONNX). Takes the crop and box and gives the shape, a position correction and the score u. It can run on TensorRT, ONNX Runtime or PyTorch; the model file type decides which.
- **Planner** (NumPy). Takes the position, shape and u, and either plans the move (approach, grasp, lift, carry, release, return home) or stops. It uses exact inverse kinematics and smooth joint paths.

- **ROS 2 nodes.** A vision node, a planning node running at 50 Hz, a simulation bridge and a viewer, started together by one launch file and connected by topics such as /camera/image_raw, /arm_target, /joint_commands, /estop and /arm_busy.
- **ESP32 firmware** (Arduino, micro-ROS). Drives the stepper motors and gripper servo, watches a hardware stop button, and stops if no command arrives for 200 ms. It sends a status message 5 times a second, so the USB link can be checked without motors attached.
- **User interfaces.** A viewer window shows the simulation, what the vision system sees and the arm's status, with keys to stop or reset. A web dashboard on the Jetson runs a fixed set of tests (self-check, speed tests, live timing, sorting with video), shows logs and board readings, and lets you download the results. It needs a login token.
- **SAC reaching test.** A separate reinforcement learning task, used only to compare learned reaching with inverse kinematics.

4.5 AI Techniques Used and Design Rationale

Task formulation

For each object, the AI part gives one of four shapes, a 3D position and an unfamiliarity score. From these, the system decides **which tray to use, or whether to stop**. The arm works on its own; a person is only needed to reset it after a stop. Errors can come from three places: the finder (missing an object or merging two), the network (wrong shape, wrong position, or an unknown object that looks familiar), and the motor controller (motor settings not yet calibrated).

Techniques used

Table 4.2: AI techniques used in the project

AI technique	Used? (Y/N)	Level*	Where in our system, and why chosen over the main alternative
Large language models and the transformer architecture (attention, tokenisation, decoding and temperature)	N		
Prompt engineering and in-context learning (zero-/few-shot, chain-of-thought, structured outputs)	N		
Fine-tuning and parameter-efficient adaptation (instruction tuning, LoRA / QLoRA)	N		
Embeddings and dense retrieval (vector search, similarity measures, hybrid search, re-ranking)	N		
Retrieval-augmented generation (chunking, grounding, citation of sources)	N		

AI technique	Used? (Y/N)	Level*	Where in our system, and why chosen over the main alternative
Knowledge graphs and GraphRAG (entity and relation extraction, graph traversal, KG-based validation)	N		
LLM agents: tool use, planning and memory (ReAct, function calling, MCP)	N		
Multi-agent coordination (role specialisation, debate, voting, deliberation)	N		
Hallucination detection and fact verification (claim decomposition, self-consistency, evidence checking)	N		
Uncertainty quantification and calibration (confidence estimation, conformal prediction, calibration error)	Y	(c)	Mahalanobis unfamiliarity score, fitted after training and tuned on validation data. Chosen over ensembles or Monte Carlo dropout because it needs only one pass through the network.
Fuzzy logic and neutrosophic reasoning (membership functions, rule inference, T/I/F components, ANFIS)	N		
Neuro-symbolic integration (rules or logic combined with neural models, symbolic justification)	Y	(c)	Fixed rules act on the network output: stop if u > 0.75, pick the tray by shape, and refuse targets the arm cannot reach. The rules are written by hand, not learned.
Bias and fairness measurement (counterfactual / paired prompting, group fairness metrics)	N		
Bias mitigation and guardrails (debiasing, input/output filtering, constrained generation)	N		
LLM evaluation methodology (benchmarks, LLM-as-a-judge, human evaluation)	N		
Adversarial attacks and defences on LLMs (jailbreaks, prompt injection, red-teaming)	N		
Multimodal models (vision-language models, speech, multimodal fusion)	N		
Generative image models (diffusion, text-to-image generation)	N		
Synthetic media detection and provenance (deepfake detection, content credentials)	N		
Long-context efficiency (context compression, summarisation, token budgeting)	N		
Agent-based modelling and simulation (information diffusion on social networks, epidemic-style models)	N		
Sequential decision-making under uncertainty (reinforcement learning, MDPs, game-theoretic strategy)	Y	(b)	SAC agent for fingertip reaching, used only as a comparison with inverse kinematics.
Few-shot learning and edge deployment (meta-learning, quantisation, on-device inference)	Y	(b)	TensorRT FP16 and FP32 engines built and timed on a Jetson Orin Nano Super. FP16 chosen over INT8 because it needs no calibration data and stays within 0.005 of the original. No few-shot learning.
Classical machine learning and computer vision (CNNs, gradient boosting, feature engineering)	Y	(b), (c)	ImageNet-pretrained MobileNetV3-small with my own output layers (b); background-subtraction object finder written by me (c).
Other (specify)	Y	(c)	Exact inverse kinematics and smooth (quintic) joint paths.

**Level: (a) used off-the-shelf; (b) configured or tuned by me; (c) modified or implemented by me.*

Technique explanations

Table 4.2 marks the techniques I used. Each one is explained below.

MobileNetV3-small with transfer learning [16]. Instead of training from scratch, I started from a network already trained on ImageNet and retrained it on my shapes. This reached 100% test accuracy after 12 epochs, while the same network without pretraining only ever guessed one class after 2 epochs. I picked MobileNetV3-small because it is built for phones and small boards. Its limit: it has only seen simulated images.

Mahalanobis unfamiliarity score [2], [15]. For each crop the network gives a feature vector f. The score is its distance to the closest of the four shape averages, $d(f) = \min_k (f - m_k)^T S^{-1} (f - m_k)$, where m_k is the average feature vector of shape k and S describes how the features vary (shared by all shapes). Unlike a plain straight-line distance, this allows for features that naturally vary a lot. I then scale the distance to a 0 to 1 range so that 0.75 matches the 99th percentile of validation distances (a raw value of 21.11). So, by design, about 1 in 100 known objects goes over the line. I chose this over Monte Carlo dropout or running several networks because it costs only one small extra calculation per object ($O(C \times D^2)$, with C = 4 shapes and D = 128 features). Its limit: an unknown object whose features land near a known shape, like the capsule near the cylinder, gets a low score.

Rule layer. Simple hand-written rules turn the network output into actions: stop if $u > 0.75$, choose the tray from the shape, and refuse any target the arm cannot reach. I used rules instead of a learned policy because their behaviour is fully predictable and easy to test (Table 7.3).

Exact inverse kinematics and smooth paths. Inverse kinematics (IK) works out the joint angles needed to put the fingertip at a given point. For this arm (a rotating base plus three joints in one vertical plane) it can be solved exactly with geometry, and my solution matches PyBullet to within 1 mm. Each move follows a fifth-order (quintic) curve, so the arm starts and stops with zero speed and zero acceleration, and no joint goes above 80% of its speed limit. I used this instead of PyBullet's built-in solver so the Jetson does not need a physics engine.

Soft Actor-Critic (SAC) [17]. SAC is a reinforcement learning method that learns by trial and error and is rewarded for exploring. My agent sees 17 numbers (joint angles and speeds, the fingertip and target positions, and the gap between them) and sets four joint speeds. I chose it because it is the standard method for smooth, continuous control. Its limit: it needs a lot of practice (300,000 steps here) and still did not match IK (Section 7.1).

5. Methodology and Implementation

5.1 Data

Table 5.1: Dataset description

Attribute	Details
Name and source (URL / citation)	Own synthetic dataset, generated by scripts/generate_sim_dataset.py in PyBullet [19]. Unseen objects: random meshes from pybullet_data/random_urdfs and a capsule shape.
Licence and terms of use	Generated by me; PyBullet and its sample meshes are under the zlib licence.
Size (samples, classes, tokens, images, documents)	6,000 known-object and 1,000 unknown-object scenes giving 18,464 crops (16,640 known, 1,824 unknown). Each crop is 128 x 128 plus box features. 4 known classes.
Collection method	Synthetic: 1 to 4 objects per scene with random colour, size (plus or minus 15%), rotation, position and one of 8 light directions; trays in view.
Train / validation / test split and how it was made	Split by scene. Known objects: 10,289 train, 2,134 validation, 2,217 test crops. Unknown objects: 918 validation (meshes 0 to 499) and 906 test (meshes 500 to 999); capsules are 25% of unknown scenes and never appear in training.
Class balance	Train: sphere 2,599, cube 2,568, cylinder 2,598, brick 2,524. Test: 563, 589, 511, 554. Colour is random and independent of class.
Personal or sensitive data present?	None. All images are rendered from simulation.

5.2 Preprocessing

Table 5.1 describes the dataset. Each sample goes through these steps before training:

- **Cropping.** Each detected object is cut out as a square, everything except the object is masked out (so tray edges and neighbours are hidden), and the crop is resized to 128 x 128. If less than 70% of a blob belongs to one object, it is treated as two merged objects and dropped.
- **Normalisation.** Images are scaled to match ImageNet colour statistics. This happens inside the model, so it cannot be forgotten at run time.
- **Augmentation** (training only). Random changes to brightness and contrast (up to 20%), colour channel order, size (up to 8%) and position (up to 4%), plus a little random noise. No flipping, because the position correction is in real-world directions.

- **Avoiding leaks.** Data is split by scene, so no scene appears in two splits. Unknown meshes for validation and testing come from two separate pools. The stop threshold is set using validation data only, and the test set is used once.

5.3 Models Used

Table 5.2: Model inventory

Model	Exact version / snapshot	Open / closed weights	Size	Role in system	Access method and date
MobileNetV3-small with my heads	ImageNet weights (torchvision); models/perception.onnx	Open	4.1 MB file	Shape label, position correction, unfamiliarity score	Local (PyTorch, ONNX Runtime); [add training date]
TensorRT engine of the same model	models/ perception_fp16.engine, TensorRT 10.3, dynamic batch 1 to 8	Open	Built on device	Same as above, on the Jetson GPU	Jetson Orin Nano Super; [add date]
SAC policy	stable-baselines3 MlpPolicy (256 x 256); models/sac_reach	Open	Small	Reaching benchmark only	Local; [add training date]

5.4 Prompting, Fine-tuning and Retrieval Configuration

Table 5.2 lists the models. There is no language model, so prompting and retrieval do not apply. **Fine-tuning:** I retrained the whole ImageNet-pretrained MobileNetV3-small for 20 epochs with two outputs: a 4-way shape classifier and a small position-correction layer (128 image features plus 4 box features, through 64 units, to 3 values in centimetres). The loss is cross-entropy for the shape (label smoothing 0.05) plus 0.5 times a Smooth L1 loss for the position. After training, I switch the network to evaluation mode and fit the score statistics on clean training images.

5.5 Experimental Setup and Hyperparameters

Table 5.3: Hyperparameters and settings

Parameter	Value(s) tried	Final value	How chosen (grid / random / Bayesian / default / literature)
Training epochs	2, 12, 20	20	Compared across runs; 20 used for the final model
Optimiser, learning rate, schedule	AdamW, 1e-3, OneCycle	Same	Common default for fine-tuning

Parameter	Value(s) tried	Final value	How chosen (grid / random / Bayesian / default / literature)
Batch size; weight decay	64; 1e-4	Same	Default
Position loss weight; label smoothing	0.5; 0.05	Same	Default
Stop threshold (percentile of validation distances)	99th	99th = 0.75	Design choice for about 1% false stops
Centre loss weight	0.3, 1.0	Not used	Both runs collapsed or diverged
SAC learning rate; batch; buffer; discount	3e-4; 256; 300k; 0.98	Same	stable-baselines3 defaults, discount lowered
SAC training steps; reward	150k (v1), 300k (v2)	300k, shaped reward	v1 plateaued at 16 cm error
TensorRT precision; batch range	FP32, FP16; 1 to 8	FP16	Fastest; same predicted classes as FP32
Jetson power mode	25 W, MAXN_SUPER	MAXN_SUPER with jetson_clocks	CPU path 14.02 ms at 25 W against 11.0 ms
Random seeds and number of runs	Sorting seeds 1000 to 1019; unknown test seeds 5000 to 5029	One training run per setting	Fixed seeds for repeatability

5.6 Compute and Cost

Item	Details
Hardware	Training: PC with Arch Linux, Intel Core i7-1360P (16 threads), 15 GB RAM, CPU only, Python 3.14. Edge: Jetson Orin Nano Super, JetPack 6 (L4T R36.5), TensorRT 10.3, CUDA 12.6, Python 3.10.12. ESP32 not yet run.
Training / indexing time	Dataset: about 4 min for 7,000 scenes. Perception: about 10 to 11 min for 20 epochs on an idle CPU. SAC: about 1.5 h for 300k steps. TensorRT engine build on the Jetson: FP16 195 s, FP32 85 s.
Inference latency (mean and p95)	Jetson, TensorRT FP16: 5.91 ms mean, 6.20 ms p95 per frame with 3 objects (finder + network + planning). Jetson CPU: 11.25 / 13.84 ms. PC CPU under shared load: 11.96 / 17.45 ms. Power on the Jetson: about 7.0 W (TensorRT) and 8.5 W (CPU).
API tokens and cost	None. All models run locally; cost Rs 0.

5.7 Tools and Frameworks

Table 5.4: Tools used and justification

Tool / library	Version	Purpose in project	Why chosen over alternatives
PyBullet	3.2.6	Physics, camera rendering, arm model	Fast, reads URDF robot files, renders without a GPU
PyTorch, torchvision	[version]	Training the perception network	Pretrained MobileNetV3 weights; ONNX export
ONNX Runtime	1.23.2 (Jetson)	Fast CPU inference, fallback on Jetson	Runs the same exported model as TensorRT
TensorRT [21], cuda-python	10.3 (JetPack 6)	GPU inference on the Jetson	Native NVIDIA engine; the aarch64 ONNX Runtime wheel is CPU-only
OpenCV, NumPy	[version]; NumPy 1.26.4 (Jetson)	Object finder, IK, planner	Small and fast; no physics engine needed on the Jetson
stable-baselines3 [20]	[version]	SAC training	Tested, standard implementation
ROS 2 Humble [18], micro-ROS	Humble	Messages between nodes and the ESP32	Standard robot middleware; micro-ROS runs on microcontrollers
Arduino IDE, AccelStepper	[version]	ESP32 firmware, stepper motion profiles	Simple toolchain I already use
Python standard library (http.server)	3.10	Edge-test dashboard	No extra packages to install on the Jetson

5.8 Implementation, Screenshots and Demo

GitHub repository: https://github.com/RPi-Boy/Automator

Demo video: Google Drive (sorting run on the Jetson)

Jetson benchmark files and video folder: Google Drive folder

The demo video shows a full sorting trial run on the Jetson with the TensorRT FP16 model. The Drive folder also holds the raw benchmark files from the Jetson. The GitHub repository has the code, trained models, PC results (models/*.json) and Jetson results (docs/jetson_results/). On the Jetson, a self-check script reports PASS, WARN or FAIL for each requirement, and a few commands build the engine, start the dashboard and launch the ROS 2 demo (Appendix B). Table 5.3 gives the settings, Section 5.6 the compute cost and Table 5.4 the tools. Results are shown in Figs. 7.1 to 7.3.

6. Evaluation Protocol

6.1 Evaluation Data and Split Hygiene

- **Perception:** 2,217 test crops of known objects, from scenes never used for training or tuning, plus 918 validation and 906 test crops of unknown objects. The two unknown sets use separate pools of meshes, and the capsule never appears in training.
- **Sorting:** 20 trials with 4 known objects each (seeds 1000 to 1019), running the full loop: camera, finder, network, planner and simulated arm.
- **Safety:** 30 trials with 3 known objects and 1 unseen random mesh (seeds 5000 to 5029).
- **Speed:** timed on real simulated frames with 3 objects: 200 runs on the PC, 500 per setup on the Jetson, and a 60 s run at 30 frames per second (about 1,800 frames) per setup. Time spent drawing the simulation is not counted.
- **Sorting on the Jetson:** 3 trials with 4 objects (seeds 1000 to 1002) using TensorRT FP16.

I reused the same sorting seeds while improving the design (Fig. 7.3), so the sorting result is not on fresh data. The perception and unknown-object tests are.

6.2 Metrics

- **Accuracy, precision, recall and macro-F1** for shape labels. F1 = 2PR / (P + R), and macro-F1 is the average over the four shapes.
- **Position error:** the distance in mm between the predicted and true centre of the object (mean and 95th percentile). This decides whether the gripper closes on the object.
- **AUROC:** the chance that a random unknown object gets a higher score than a random known one. 1.0 is perfect and 0.5 is guessing. It shows how well the score separates the two groups without picking a threshold.
- **Detection rate at 0.75** (share of unknowns caught) and **FPR at 95% TPR** (share of known objects that would be stopped if the threshold were set to catch 95% of unknowns).
- **False-stop rate:** share of known test crops with u above 0.75.
- **Sorting success:** share of known objects that end up in their own tray. **Stop rate:** share of trials that stopped on the unseen object.
- **Speed:** mean, median, 95th percentile and worst time per frame; frames per second = 1000 / mean time in ms. **Power and load:** average watts and CPU and GPU use, read from tegrastats.
- **SAC success:** share of episodes where the fingertip ends within 1 cm of the target.

6.3 Baselines

- **Simple baselines:** always guessing the most common shape (cube) gives 26.6% accuracy; a random score gives an AUROC of 0.50.
- **Simple position method:** camera geometry alone (the blob centre projected onto the table), with no learned correction.
- **Original project:** the first version of this project, before I rebuilt it, run with its own test.
- **Strong baseline:** the same network trained without masked crops and without trays in the data (model C). It differs from the final model only in that one fix.
- **Control baseline:** exact IK against the learned SAC agent for reaching.
- **Hardware baseline:** ONNX Runtime on the Jetson CPU against TensorRT on its GPU, with the same model and frames.

I did not train an established model from the literature (such as YOLOv10 [10]) on this data. This gap is noted in Section 7.6.

6.4 Human Evaluation and LLM-as-a-Judge (if used)

Not applicable. No person labelled any data and no language model was used to judge results; all labels come from the simulator.

6.5 Statistical Analysis

Each setup was trained once, with one seed, so I cannot report how results vary between training runs. For percentages I give 95% Wilson confidence intervals. For sorting, I also resampled the 20 trials 10,000 times (bootstrap) to get a confidence interval. To compare sorting between versions I used Fisher's exact test on objects sorted versus not sorted. Since I ran two tests, I used a stricter significance level of 0.025 (Bonferroni correction).

7. Results and Discussion

7.1 Main Results

Table 7.1: Main results (single training run; 95% CI in brackets; best value in bold)

Method	Shape accuracy	Position error, mean (mm)	Unknown AUROC (test)	Sorting success	Time per frame (ms)
Naive (majority class / random score)	26.6%	Not applicable	0.50	Not applicable	Not applicable
Original project (before rework)	Not valid (trained on noise images)	Not measured	Not measured	0.0% (placement error 0.363 m)	Claimed 10.28 ms, not valid
Model C: unmasked crops (strong)	**100%**	**1.84**	0.926	80.0% (32/40)	Not measured
Ours (final, model D)	99.91% [99.7, 100.0]	1.97 (geometry only: 5.43)	**0.934**	**92.5% (74/80)** [84.6, 96.5]	**5.91 mean, 6.20 p95** (Jetson, TensorRT FP16)

Table 7.1 compares the final system with the baselines. The network almost never gets the shape wrong: 2 mistakes in 2,217 crops (Fig. 7.1). Precision is 100% for sphere, cube and cylinder and 99.64% for brick; recall is 100%, 99.83%, 99.80% and 100%; macro-F1 is 99.91%. The learned correction cuts the average position error from 5.43 mm (camera geometry only) to 1.97 mm, and the 95th percentile from 10.98 mm to 4.17 mm, a 64% drop. Only 0.72% of known crops went over the stop threshold (16 of 2,217; 95% CI 0.4% to 1.2%), in line with the 1% it was tuned for.

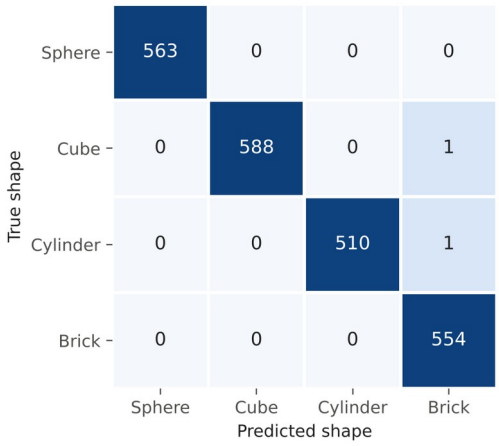


Fig. 7.1: Confusion matrix on 2,217 held-out crops

Table 7.2 breaks the unknown-object results down by type. Random meshes, which look nothing like the training shapes, are almost always caught (AUROC 0.999, 95.4% flagged). The capsule, which is basically a cylinder with rounded ends, mostly is not (13.6% flagged). Since capsules are a quarter of the unknown set, the overall AUROC is 0.934 and the FPR at 95% TPR is 15.6%, so both targets are missed.

Table 7.2: Unknown-object detection by split and object type

Split	n	AUROC (all)	Flagged at 0.75	FPR at 95% TPR	Meshes: AUROC / flagged	Capsule: AUROC / flagged
Validation	918	0.931	76.4%	16.2%	0.999 / 97.1%	0.728 / 14.7%
Test	906	0.934	76.8%	15.6%	0.999 / 95.4%	0.714 / 13.6%

Table 7.3 shows the safety test with the full system running. The arm stopped in every trial where the finder saw the unknown object, and it never stopped because of a known object.

Table 7.3: Safety test with one unseen object among three known objects

Outcome (30 trials)	Count	Comment
Unknown object detected by the finder	25 / 30	Missed in 5 trials (Section 7.4)
Detected unknown flagged (u > 0.75)	25 / 25	Every detected unknown was flagged
Trials with an emergency stop	25 / 30	All 25 stops were on the unknown object; 0 on known objects
Trials where the arm moved the unknown object	1 / 30	Seed 5024: object moved before the stop
Known objects sorted	17 / 90	Low by design: the arm stops before sorting when it sees an unknown

Table 7.4 and Fig. 7.2 show the time per frame for each setup (500 runs each on the Jetson, in its fastest power mode, MAXN_SUPER). Every setup fits within 30 ms. On the Jetson, TensorRT FP16 makes the network step 3.2 times faster (5.94 to 1.84 ms) and the whole step 1.9 times faster (11.25 to 5.91 ms, about 169 frames per second). It is also much steadier: the slowest frame drops from 27.6 ms to 6.8 ms. FP16 is only slightly faster than FP32 for a network this small, and its outputs match FP32 to within 0.005 with the same shape every time. The GPU work itself takes only 0.89 ms (measured with trtexec); the rest of the 1.84 ms is copying data and Python overhead. In the board's default 25 W mode, the CPU version took 14.02 ms.

Table 7.4: Time per frame for each setup (ms, 3 objects per frame)

Setup	Finder	Network	Plan	Total mean	p95	Max
PC CPU, ONNX Runtime (shared load, 200 runs)	5.39	5.77	0.80	11.96	17.45	20.01
Jetson CPU, ONNX Runtime	4.14	5.94	1.17	11.25	13.84	27.61
Jetson GPU, TensorRT FP32	3.04	2.04	1.08	6.16	6.48	6.80
Jetson GPU, TensorRT FP16	3.00	**1.84**	1.07	**5.91**	**6.20**	6.83

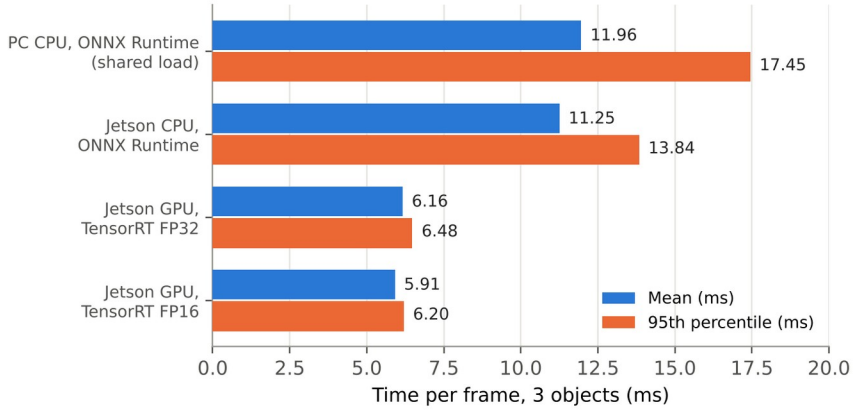


Fig. 7.2: Time per frame for each setup, mean and 95th percentile

Table 7.5 shows a 60 s run at 30 frames per second. Neither setup slowed down over time, and TensorRT used less power (7.0 W against 8.5 W) while leaving the CPU nearly idle. I also ran the sorting test on the Jetson with TensorRT FP16: **all 12 objects** went to the right tray, with no stops (seeds 1000 to 1002). This run took 711 s, but only because drawing the simulated camera image takes 270 to 350 ms per frame on the Jetson's CPU; the vision step itself took about 5.6 ms. This is the run shown in the demo video.

Table 7.5: Sustained 60 s test on the Jetson Orin Nano Super

Setup (60 s at 30 Hz)	Mean	p95	Max	First vs second half	CPU use	Power avg (max)	Temp.
Jetson GPU, TensorRT FP16	6.03 ms	6.31	6.72	6.01 to 6.04	7%	**7.0 W** (7.7)	48 C
Jetson CPU, ONNX Runtime	10.17 ms	10.63	17.84	10.18 to 10.15	44%	8.5 W (8.7)	47 to 49 C

Reaching (SAC against IK). In 100 new episodes, the best SAC agent got the fingertip within 1 cm of the target only 16% of the time, with an average final error of 18.4 mm (95th percentile

28.0 mm). When it did succeed, it took 69 of the 150 steps. IK reached all 300 tested targets to within 1 mm. SAC was still improving when training stopped at 300,000 steps.

7.2 Comparative Analysis

Against the baselines. The final model is far ahead of the simple baselines (26.6% accuracy, AUROC 0.50) and of the original project (0% sorted). Compared with the strong baseline (model C), it gives up a tiny amount of accuracy (100% to 99.91%, which is 2 crops) and position error (1.84 to 1.97 mm), but separates unknown objects better (AUROC 0.926 to 0.934) and stops by mistake less often (1.08% to 0.72%). Sorting went from 43.8% (35 of 80) in the first full 20-trial run to 92.5% (74 of 80). Fisher's exact test gives $p < 0.001$, though I changed several things between those two runs. The last two versions (87.5% and 92.5%) are not significantly different ($p = 0.43$), so I cannot claim the last change helped. Resampling the final run gives a 95% interval of 86.3% to 97.5%.

On the edge board. TensorRT is about twice as fast as either CPU and much more consistent (95th percentile 6.20 ms against 13.84 ms). The first version of this project claimed 10.28 ms and predicted 1.2 ms with INT8, but never measured either; the measured 5.91 ms replaces both.

Against the literature. My 92.5% sorting rate is close to AnyGrasp (93.3%) [6] and Sun et al. (94%) [7], but my shapes are simple and simulated, so this is not a fair head-to-head comparison. My overall AUROC of 0.934 is higher than the best result on hard ImageNet unknowns in OpenOOD v1.5 (82.16%) [1], again on an easier task. The capsule result agrees with [1] and [2]: unknown objects that look like a known class are the hardest to catch.

7.3 Ablation Study

Table 7.6: Ablation, effect of removing each component

Configuration	Metric	Value	Change vs full system
Full system	Sorting; false stops; position error	92.5%; 0.72%; 1.97 mm	Not applicable
Without learned position correction	Position error (mean)	5.43 mm	+3.46 mm (2.8x worse)
Without masked crops and trays in training (model C)	AUROC; false stops	0.926; 1.08%	-0.008; +0.36 points
Without masked crops, trays in view (single objects)	False alarms (unmasked model)	2.0% overall; 7.5% in one image band	From 0.3% without trays

Configuration	Metric	Value	Change vs full system
Without rolling friction, rimmed trays and low release	Sorting	43.8% (35/80)	-48.7 points
Without pausing vision while the arm moves	Checks with u > 0.75 in 3 runs	0, 0 and 4 checks	Arm links flagged as unknown
Without spacing objects apart in the image	Merged blobs flagged	10 of 10	Every merged blob stops the arm
With centre loss added (weight 0.3)	Shape accuracy; AUROC	25.4%; 0.50	Features collapsed; dropped
Without ImageNet pretraining (2 epochs)	Shape accuracy	25.4%	Predicts one class
Without TensorRT (Jetson CPU, ONNX Runtime)	Time per frame, mean / p95	11.25 / 13.84 ms	+5.34 / +7.64 ms

Each row in Table 7.6 is a measured run that differs from the full system mainly in the part named. The biggest effects come from the simulation physics (objects rolling away) and the learned position correction. Masking the crops mostly reduces false stops, and TensorRT halves the time per frame. Fig. 7.3 shows how sorting improved as each fix was added.

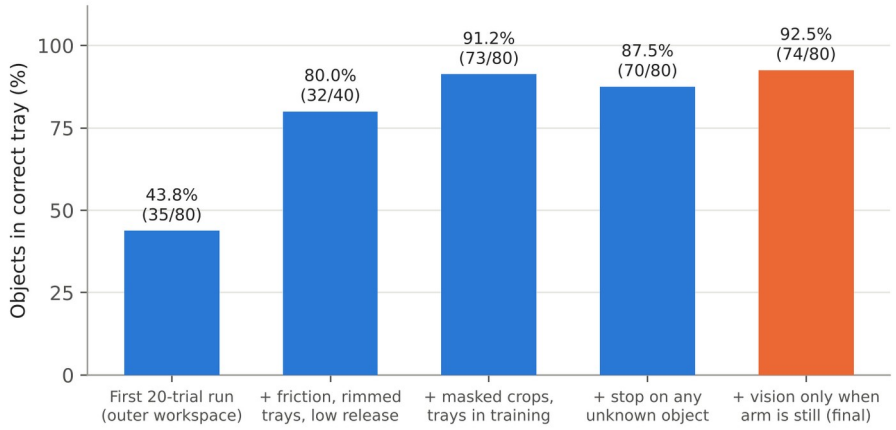


Fig. 7.3: Sorting success as each change was added

7.4 Error Analysis

Table 7.7 lists the failure cases. Grouped by cause: **landing outside the tray** (cases 2 and 3, 5 of the 6 sorting failures), **wrong shape** (case 1, 1 of 6), **object not found** (case 4, 5 of 30 safety trials), **unknown objects that look known** (case 6) and **merged objects** (case 7). Missed objects matter most for safety: if the finder never sees an object, the network never gets a chance to flag it.

Table 7.7: Representative failure cases

#	Input	Output	Expected	Diagnosis
1	Sort, seed 1013: brick	Brick in sphere tray (43.7 cm from its own tray, 1.4 cm from sphere tray)	Brick tray	Shape error: brick labelled as sphere
2	Sort, seeds 1004, 1008, 1015, 1017	Sphere, cube, cube, brick ended 9.9 to 10.9 cm from tray centre	Inside tray (radius 6.5 cm)	Landed just outside the rim; probably bounced or was dropped early (cause not confirmed)
3	Sort, seed 1013: sphere	Ended 13.2 cm from its tray	Sphere tray	Possibly knocked by the arm while it handled the brick (cause not confirmed)
4	Safety, seeds 5007, 5014, 5018, 5022, 5025	No stop; arm sorted the 3 known objects	Stop	Finder never saw the mesh, probably low contrast or very flat
5	Safety, seed 5024	Unknown object moved, then stop	Stop before contact	The unknown object, outside the pick area, was moved during a pick
6	Unknown test set: capsules	86.4% of capsules scored u < 0.75	Flagged	Looks too much like a cylinder to the network
7	Overlapping objects	Merged blob flagged (10 of 10)	Two known objects	Finder cannot separate touching objects

7.5 Answers to the Research Questions

- **H1: supported.** Accuracy was 99.91% (target 99%) and mean position error 1.97 mm (target 5 mm).
- **H2: partly supported.** The threshold behaves as tuned (0.72% false stops), and random unknown meshes are caught almost perfectly (AUROC 0.999). Capsules are not, so the overall AUROC (0.934) and FPR at 95% TPR (15.6%) miss their targets.
- **H3: supported in simulation, with one gap.** 92.5% of known objects were sorted. The arm stopped in all 25 trials where the unknown object was seen and never on a known object, but it did not stop in the 5 trials where the finder missed the object.
- **H4: not supported.** SAC reached 1 cm in only 16% of episodes, while IK was within 1 mm every time.
- **H5: supported.** On the Jetson Orin Nano Super the step takes 5.91 ms with TensorRT FP16 (95th percentile 6.20 ms), compared with 11.25 ms on its CPU, well inside 30 ms.

7.6 Threats to Validity and Limitations

- **Internal validity.** Each setup was trained once; the sorting seeds were reused during development; and I did not find the cause of 5 of the 6 sorting failures. In the first version, a test with a loose pass mark hid a 0% success rate, so I rewrote all the tests (all 21 now pass).
- **External validity.** All images are simulated, so there is no real camera noise. The gripper holds objects by locking them in place, not by real finger contact. There are only four simple shapes and one kind of look-alike unknown, and I did not separately test how image noise affects the results. The results may not carry over to the real IMX219 camera or arm, and the ROS 2 nodes, ESP32 and arm have not been run yet.
- **Construct validity.** The unknown-object scores depend on which unknowns are chosen; because a quarter of them are capsules, they pull the overall AUROC down. Jetson timing does not include drawing the simulation, and it was measured in only one power mode for 60 s, which is too short to show heat effects.
- **Missing baseline.** I did not train an established detector from the literature on the same data.

8. Responsible AI and Ethical Considerations

8.1 Risk Assessment

Table 8.1 lists the main risks, who they could affect and what is already in place.

Table 8.1: Risks and mitigations

Risk	Who could be harmed	Likelihood / severity	Mitigation implemented
Unknown object (or hand) not detected, arm moves it	People and items near the arm	Medium (5/30 misses in sim) / High	Hardware stop pin, 200 ms watchdog; physical guard needed before real use
Look-alike unknown accepted as known	Items mistaken for a known shape	Medium / Medium	Reported as open; feature normalisation [2] and multi-layer scores [3] planned
Arm sags when stopped (drivers off)	People under the arm; hardware	Medium / Medium	Documented; needs brakes or a safe stop pose
Uncalibrated stepper ratios	Hardware, bystanders	High if powered now / Medium	Flagged in code; calibrate before powering
Trusting simulation results too much	Readers, reviewers	Medium / Low	Every number labelled as simulation, PC or Jetson; untested hardware marked as such
Leaked credentials	Project owner	Low / Medium	client_secrets.json git-ignored; to be rotated
Misuse of the Jetson dashboard	Jetson owner, network users	Low / Medium	Token login, fixed job list, no free commands; keep on a trusted network

8.2 Data Privacy and Consent

No personal data is collected; all training and test images come from the simulator. The real IMX219 camera could see people near the arm, so its frames should be processed in memory and not saved. No people took part in this study, so consent and the Digital Personal Data Protection Act, 2023 do not apply to the current work.

8.3 Fairness

No people or groups are involved. The fairness question here is whether the system works equally well for every object type and look. Recall ranges from 99.80% to 100% across the four shapes. Colour is random and unrelated to shape, and 8 lighting directions are used, so the network cannot cheat by relying on colour or lighting. Real lighting and real materials have not been tested.

8.4 Safety, Security and Misuse

The main safeguard is the stop chain: a high score, a stop message, no command for 200 ms, or the hardware button all trigger the same halt, which stays on until a reset message is sent and the button is released. One known security gap is that ROS 2 messages are not authenticated, so

anyone on the same network could send a reset or movement command; SROS2 security should be turned on before real use. The Jetson dashboard only runs a fixed list of jobs, needs a token, and should stay on a trusted network. A small sorting arm has little potential for misuse.

8.5 Licences and Attribution

PyBullet and its sample meshes (zlib), PyTorch and torchvision (BSD-3-Clause), MobileNetV3 ImageNet weights from torchvision, ONNX Runtime (MIT), OpenCV (Apache 2.0), stable-baselines3 (MIT), ROS 2 and micro-ROS (Apache 2.0), AccelStepper (GPL-3.0, firmware only), TensorRT and cuda-python (NVIDIA licence terms, used as installed with JetPack). Repository licence: [choose, for example MIT for code outside the firmware].

9. Conclusion and Future Work

Achieved. Against the objectives: shape accuracy 99.91% (O1, met); position error 1.97 mm (O2, met); unknown-object AUROC 0.934 and FPR 15.6% (O3, not met; 0.999 on random meshes, 0.714 on capsules); false stops 0.72% (O4, met); 92.5% of objects sorted (O5); a stop in 25 of 30 unknown-object trials, every time the object was seen (O6, partly met); SAC within 1 cm in 16% of episodes (O7, not met); 5.91 ms per frame (95th percentile 6.20 ms) on the Jetson with TensorRT FP16, at about 7 W (O8, met).

Main lesson. Most of the improvement came from fixing the data and the surrounding system, not from a bigger network. Masking the crops, realistic physics, pausing vision while the arm moves, and honest tests mattered more than the model itself. The unfamiliarity score can only judge objects the finder actually sees.

Future work.

- Run the ROS 2 nodes live on the Jetson, and do longer runs in the 15 W and 25 W power modes.
- Compile and flash the ESP32 firmware, calibrate the motors, and test the hardware stop on the real arm.
- Catch look-alike objects better by normalising features [2] and using scores from several network layers [3], and add capsule-like shapes to validation.
- Miss fewer objects by adding a depth check or an open-set detector [11], and train a YOLOv10 [10] baseline on the same data.
- Train SAC for 1 to 2 million steps, and repeat the key runs with at least 3 seeds.

10. Reproducibility Checklist

Item	Yes / No / NA	Location (file or section)
Repository shared with the faculty; README has setup and run steps	Yes	README.md; github.com/RPi-Boy/Automator
Dependencies pinned	Partly	requirements.txt; scripts/jetson_install.sh
Exact model identifiers and experiment dates recorded	Partly	Section 5.3; models/; docs/jetson_results/
All prompts included	NA	No language models used
Random seeds fixed and reported	Yes	Sections 5.5 and 6.1
Evaluation set or script to regenerate it provided	Yes	scripts/ generate_sim_dataset.py
One script reproduces the main results table	Partly	Seven commands, Appendix B
Data sources and licences listed	Yes	Sections 5.1 and 8.5
No API keys, passwords or personal data in the repository	Yes	.gitignore (client_secrets.json)
Compute and API cost reported	Yes	Section 5.6
Demo video link opens without a permission request	Yes	Section 5.8 (Google Drive, shared link)

11. References

[1] J. Zhang et al., "OpenOOD v1.5: Enhanced benchmark for out-of-distribution detection," J. Data-centric Machine Learning Research, 2024, arXiv:2306.09301.

[2] M. Müller and M. Hein, "Mahalanobis++: Improving OOD detection via feature normalization," in Proc. 42nd Int. Conf. Machine Learning (ICML), PMLR 267, 2025, arXiv:2505.18032.

[3] H. Anthony and K. Kamnitsas, "On the use of Mahalanobis distance for out-of-distribution detection with neural networks for medical imaging," arXiv preprint arXiv:2309.01488, 2023.

[4] Y. Shi, E. Welte, M. Gilles, and R. Rayyes, "vMF-Contact: Uncertainty-aware evidential learning for probabilistic contact-grasp in noisy clutter," arXiv preprint arXiv:2411.03591, 2024.

[5] A. Z. Ren et al., "Robots that ask for help: Uncertainty alignment for large language model planners," in Proc. Conf. Robot Learning (CoRL), PMLR 229, 2023, arXiv:2307.01928.

[6] H.-S. Fang et al., "AnyGrasp: Robust and efficient grasp perception in spatial and temporal domains," IEEE Trans. Robotics, vol. 39, no. 5, pp. 3929-3945, 2023, doi: 10.1109/TRO.2023.3281153.

[7] R. Sun, C. Wu, X. Zhao, B. Zhao, and Y. Jiang, "Object recognition and grasping for collaborative robots based on vision," Sensors, vol. 24, no. 1, art. 195, 2024, doi: 10.3390/s24010195.

[8] Y. Goorun, S. Marais, and F. Nicolls, "Robotic waste sorting using deep learning," MATEC Web of Conferences, vol. 406, art. 10003, 2024, doi: 10.1051/matecconf/202440610003.

[9] D. Qin et al., "MobileNetV4: Universal models for the mobile ecosystem," in Proc. European Conf. Computer Vision (ECCV), 2024, arXiv:2404.10518.

[10] A. Wang et al., "YOLOv10: Real-time end-to-end object detection," in Advances in Neural Information Processing Systems (NeurIPS), 2024, arXiv:2405.14458.

[11] S. Liu et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in Proc. European Conf. Computer Vision (ECCV), 2024, arXiv:2303.05499.

[12] B. Zitkovich et al., "RT-2: Vision-language-action models transfer web knowledge to robotic control," in Proc. Conf. Robot Learning (CoRL), PMLR 229, 2023, arXiv:2307.15818.

[13] C. Tang et al., "Deep reinforcement learning for robotics: A survey of real-world successes," in Proc. AAAI Conf. Artificial Intelligence, 2025, arXiv:2408.03539.

[14] T. Pandi, A. K. Inkulu, J. K. Raghupatruni, P. K. V. Tirumalaraju, and M. V. A. R. Bahubalendruni, "Experimental validation of jerk-minimized trajectory planning in a 4-DOF industrial robotic arm using inverse kinematics with root multiplicity," Industrial Robot, 2026, doi: 10.1108/IR-11-2025-0397.

[15] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," in Advances in Neural Information Processing Systems (NeurIPS), 2018, arXiv:1807.03888.

[16] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2019, arXiv:1905.02244.

[17] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in Proc. Int. Conf. Machine Learning (ICML), 2018, arXiv:1801.01290.

[18] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics, vol. 7, no. 66, 2022, doi: 10.1126/scirobotics.abm6074.

[19] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016-2021. [Online]. Available: https://pybullet.org (accessed Oct. 2026).

[20] A. Raffin et al., "Stable-Baselines3: Reliable reinforcement learning implementations," J. Machine Learning Research, vol. 22, no. 268, pp. 1-8, 2021.

[21] NVIDIA, "TensorRT," software. [Online]. Available: https://developer.nvidia.com/tensorrt (accessed Oct. 2026).

12. Declaration of Generative AI Use

This section covers AI tools used to produce the project and report. AI models that are components of the system are described in Section 5.3. Table 12.1 lists the tools used.

Table 12.1: Generative AI tools used

Tool and version	Purpose	Files / sections affected	Extent (minor edits / substantial draft / generated code)	How the output was verified
Claude (Anthropic), via Claude Code	Auditing the first version, rewriting code, debugging, running experiments, documentation	perception/, sim/, scripts/, ros2_ws/, firmware/, tests/, webapp/, docs/	Substantial generated code	Re-ran all experiments and the 21-test suite; checked every number against the result files
Claude (Anthropic)	Drafting, formatting and simplifying the wording of this report from my project documents	All sections of this report	Substantial draft	Checked all numbers against models/*.json and docs/JETSON_RESULTS.md; opened every reference
Antigravity IDE	Early prototype code (first review)	First prototype	Generated code	Replaced during the rework
Gemini	Ideation and wording for the first review report	First review report	Minor edits	Read and revised by me

I confirm that I have reviewed, tested and understood all AI-assisted content; that every reference was checked against its original source; and that I take full responsibility for the accuracy and integrity of this report and code.

Appendices

Appendix A. Not applicable: no prompts are used, as the system contains no language model.

Appendix B. Per-trial sorting results (Table B.1) and the commands that reproduce every result in Section 7.

Table B.1: Sorting results by trial (seeds 1000 to 1019)

Trial seeds	Objects	Sorted	Wrong or off tray	Stops
1000-1003, 1005-1007, 1009-1012, 1014, 1016, 1018, 1019	4 each (56)	56	0	0
1004, 1008, 1015, 1017	4 each (16)	12	4	0
1013	4	2	2	0
Total (20 trials)	**80**	**74**	**6**	**0**

python scripts/generate_sim_dataset.py (dataset, about 4 min)
python scripts/train_perception.py --pretrained (model and perception_metrics.json)
python scripts/sort_demo.py --trials 20 --objects 4 --video models/sort_demo.mp4
python scripts/sort_demo.py --trials 30 --objects 4 --unknown 1 --seed 5000 --out models/sort_unknown_results.json
python scripts/train_sac_reach.py --timesteps 300000 --out models/sac_reach
python scripts/benchmark_edge.py --model models/perception.onnx --tag dev_pc_onnx_cpu
python -m unittest discover tests (21 tests)

On the Jetson:
scripts/jetson_install.sh; python3 scripts/jetson_selfcheck.py; scripts/build_trt_engine.sh fp16
python3 scripts/benchmark_edge.py --model models/perception_fp16.engine --tag jetson_trt_fp16
scripts/start_dashboard.sh (browser dashboard)
scripts/run_jetson_demo.sh model_path:=$PWD/models/perception_fp16.engine (ROS 2 demo, not yet run)

Appendix C. Not applicable: no user study.

Appendix D. Not applicable: labels come from the simulator, with no human annotation.